

AI ASSISTED CODING (ASSIGNMENT-3.1)

NAME: S.Vishnu Teja

BATCH NO: 24

ROLL NO: 2503A52720

Q1. Zero-Shot Prompting (Palindrome Number Program)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.

Task:

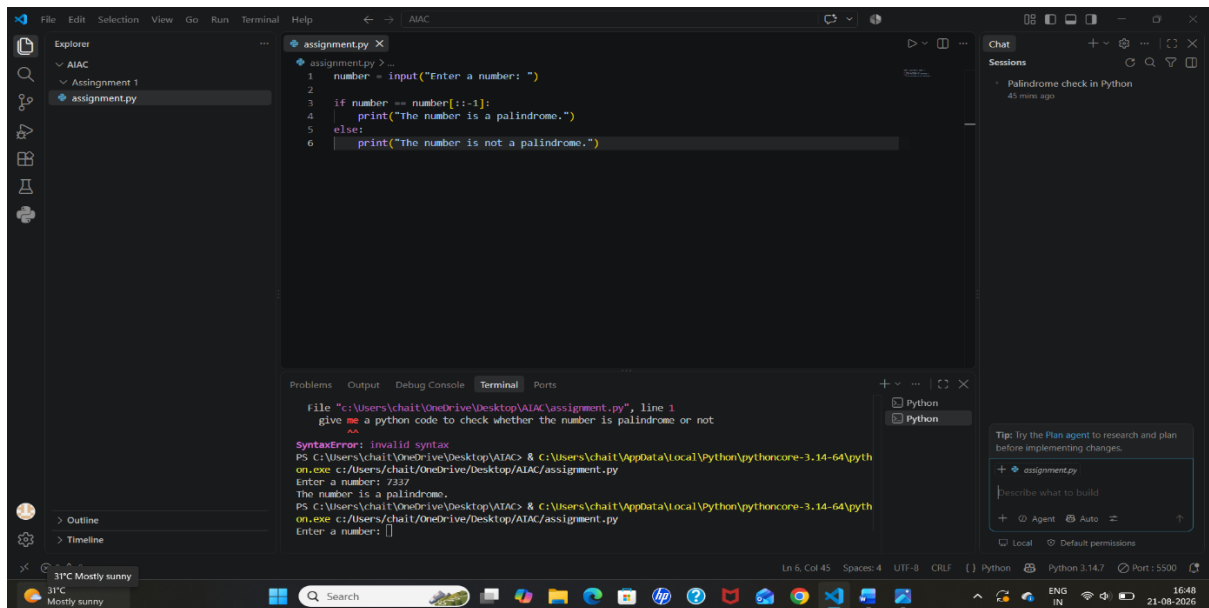
- Record the AI-generated code.
- Test the code with multiple inputs.
- Identify any logical errors or missing edge-case handling.

Prompt: give a python code to check whether the number is palindrome or not

Code:

```
number = input("Enter a number: ")  
  
if number == number[::-1]:  
    print("The number is a palindrome.")  
else:  
    print("The number is not a palindrome.")
```

output:



Q2. One-Shot Prompting (Factorial Calculation)

Write a one-shot prompt by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a given number.

Example:

Input: 5 → Output: 120

Task:

- Compare the generated code with a zero-shot solution.
- Examine improvements in clarity and correctness.

Prompt: generate a python code to calculate the factorial of a given number

Code:

```
number = int(input())

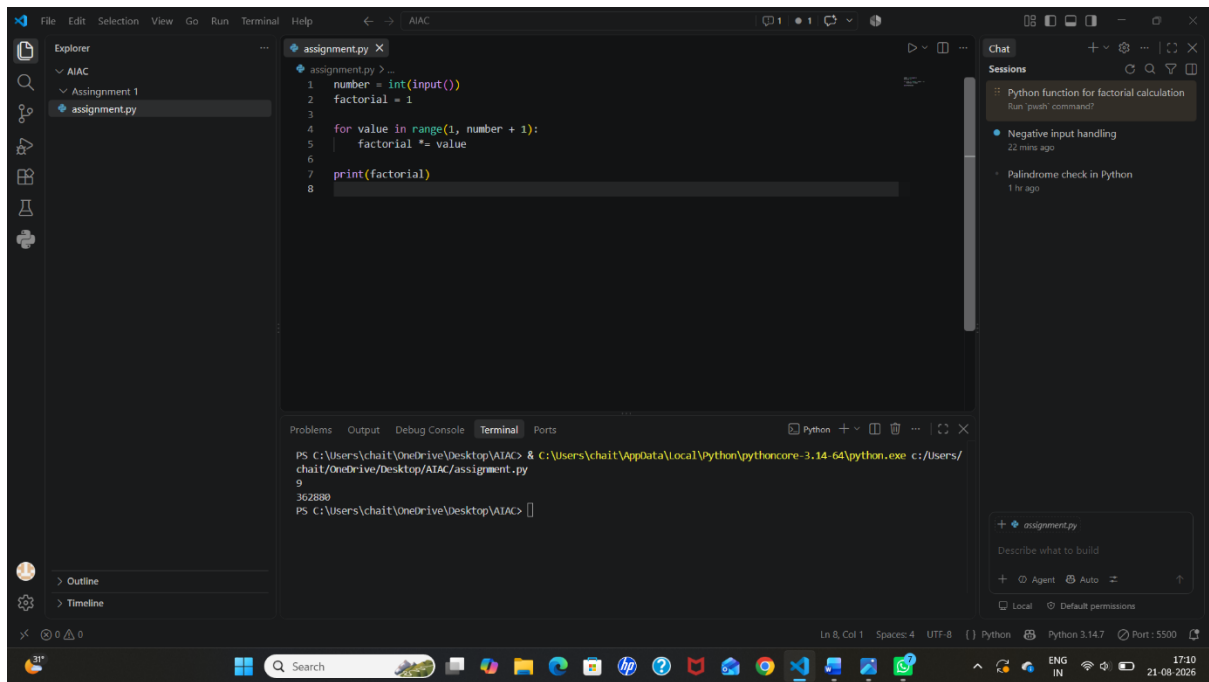
factorial = 1

for value in range(1, number + 1):

    factorial *= value

print(factorial)
```

output:



Q3. Few-Shot Prompting (Armstrong Number Check)

Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python function to check whether a given number is an Armstrong number.

Examples:

- Input: 153 → Output: Armstrong Number
- Input: 370 → Output: Armstrong Number
- Input: 123 → Output: Not an Armstrong Number

Task:

- Analyze how multiple examples influence code structure and accuracy.
- Test the function with boundary values and invalid inputs.

Prompt: give a python code to check whether a number is Armstrong number or not.

Code:

```
number = int(input("Enter a number: "))

digits = str(number)

power = len(digits)

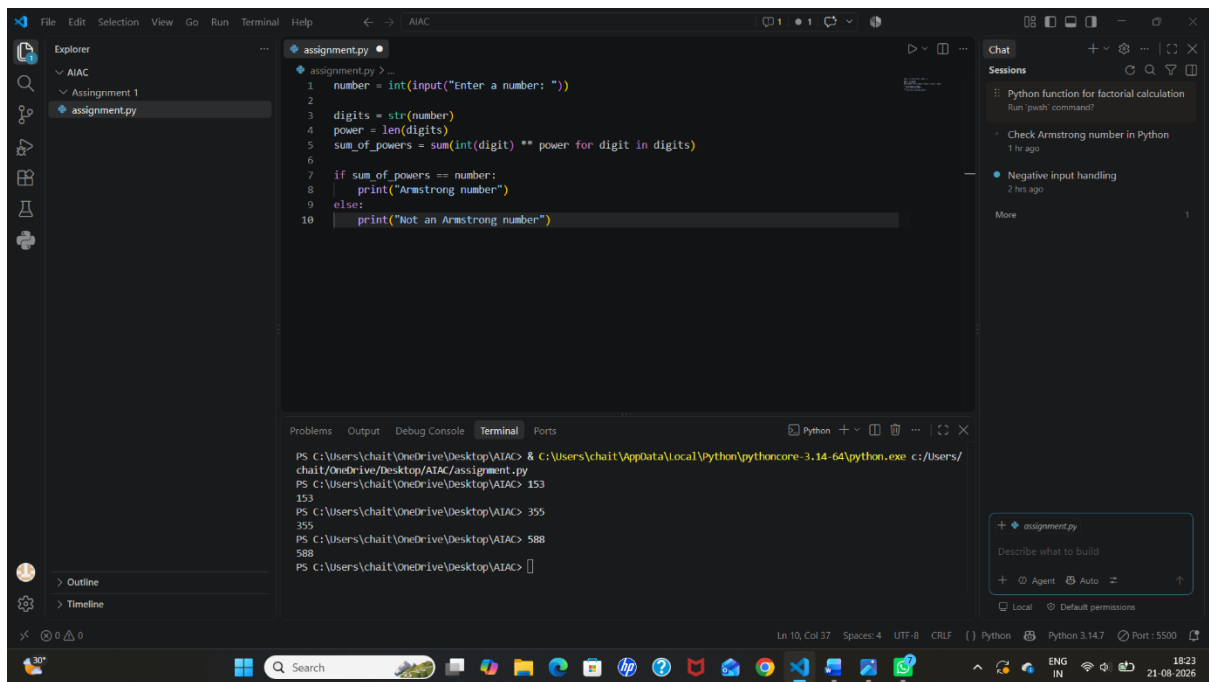
sum_of_powers = sum(int(digit) ** power for digit in digits)

if sum_of_powers == number:

    print("Armstrong number")

else:
```

```
print("Not an Armstrong number")
```



Q4. Context-Managed Prompting (Optimized Number Classification)

Design a context-managed prompt with clear instructions and constraints to generate an optimized Python program that classifies a number as prime, composite, or neither.

Task:

- Ensure proper input validation.
- Optimize the logic for efficiency.

Compare the output with earlier prompting strategies

Prompt: give a python code to classifies a number as prime, composite or neither

Code:

```
import math

def classify_number(number):

    if number < 2:

        return "neither"

    for divisor in range(2, math.isqrt(number) + 1):

        if number % divisor == 0:

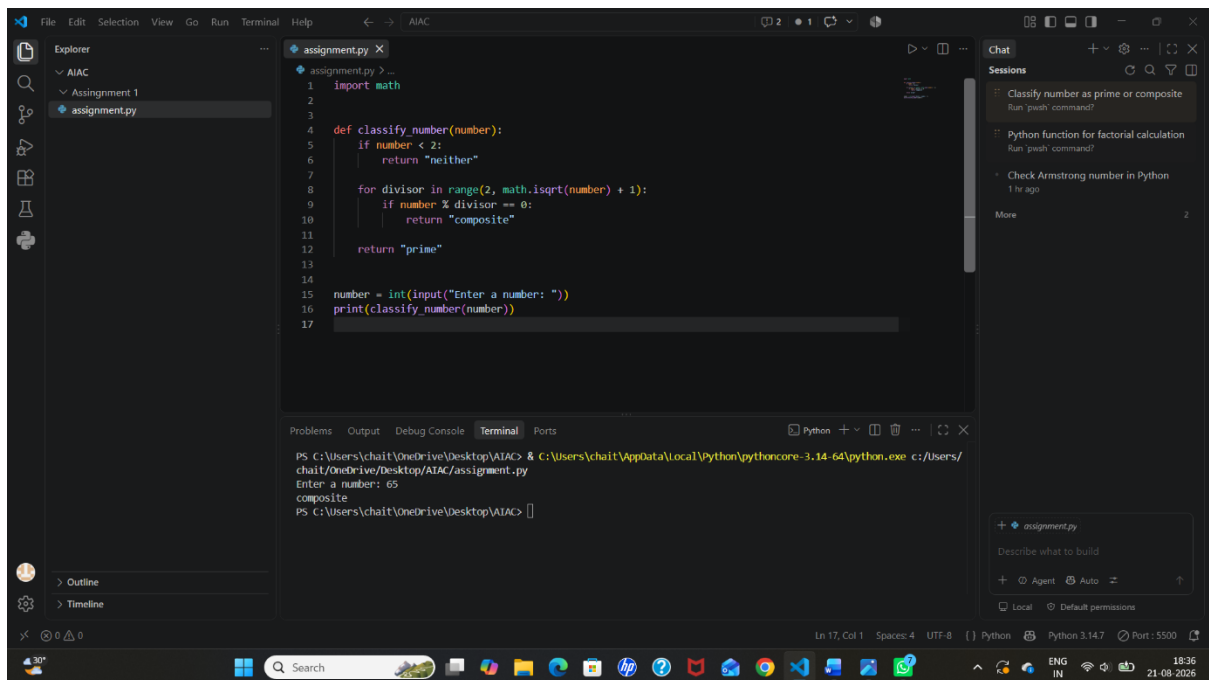
            return "composite"

    return "prime"

number = int(input("Enter a number: "))

print(classify_number(number))
```

output:



The screenshot shows a Visual Studio Code editor with a file named `assignment.py` open. The code defines a function `classify_number` that checks if a number is prime or composite. It uses a loop to check divisors from 2 to $\sqrt{\text{number}}$. The main part of the script prompts the user to enter a number and prints the result. The terminal at the bottom shows the command prompt running the script, entering the number 65, and receiving the output 'composite'.

```
1 import math
2
3
4 def classify_number(number):
5     if number < 2:
6         return "neither"
7
8     for divisor in range(2, math.isqrt(number) + 1):
9         if number % divisor == 0:
10            return "composite"
11
12    return "prime"
13
14
15 number = int(input("Enter a number: "))
16 print(classify_number(number))
17
```

Terminal Output:

```
PS C:\Users\chait\OneDrive\Desktop\AIAC> & C:\Users\chait\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/
chait/OneDrive/Desktop/AIAC/assignment.py
Enter a number: 65
composite
PS C:\Users\chait\OneDrive\Desktop\AIAC>
```

Q5. Zero-Shot Prompting (Perfect Number Check)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a perfect number.

Task:

- Record the AI-generated code.
- Test the program with multiple inputs.
- Identify any missing conditions or inefficiencies in the logic.

Prompt: give a python code to check whether a given number is a perfect number or not.

Code:

```
number = int(input("Enter a number: "))

divisor_sum = sum(
    divisor for divisor in range(1, number)
    if number % divisor == 0
)

if divisor_sum == number:
    print("Perfect number")
else:
    print("Not a perfect number")
```

output:

The image shows a Visual Studio Code editor window with a Python script named `assignment.py` open. The script checks if a number is a perfect number by summing its divisors. The terminal shows the script being executed with the input `8`, resulting in the output `Perfect number`. The AI chat sidebar on the right suggests related tasks like classifying numbers as prime or composite and checking for perfect numbers in Python.

```
1 number = int(input("Enter a number: "))
2
3 divisor_sum = sum(
4     divisor for divisor in range(1, number)
5     if number % divisor == 0
6 )
7
8 if divisor_sum == number:
9     print("Perfect number")
10 else:
11     print("Not a perfect number")
```

Terminal Output:

```
PS C:\Users\chait\OneDrive\Desktop\AIAC> & C:\Users\chait\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/chait/OneDrive/Desktop/AIAC/assignment.py
PS C:\Users\chait\OneDrive\Desktop\AIAC> 8
PS C:\Users\chait\OneDrive\Desktop\AIAC>
```

Chat Suggestions:

- Classify number as prime or composite
Run 'push' command?
- Python function for factorial calculation
Run 'push' command?
- Check perfect number in Python
6 mins ago